

LEARNING TO RANK

Learning to Rank is a class of techniques that apply supervised machine learning to solve ranking problems. It is widely used in information retrieval (IR), natural language processing (NLP), and recommender systems.

[MACHINE LEARNING](#) [RANKING](#) [RECOMMENDER SYSTEMS](#)

AUTHOR
DANIEL FAT

PUBLISHED
APRIL 23, 2021



Imports

► Code

What is Ranking ?

Ranking of query is one of the fundamental problems in information retrieval (IR), the scientific/engineering discipline behind search engines.

Given a query q and a collection D of documents that match the query, the problem is to rank, that is, sort, the documents in D according to some criterion so that the "best" results appear early in the result list displayed to the user.

Ranking in terms of information retrieval is an important concept in computer science and is used in many different applications such as search engine queries and recommender systems.

A majority of search engines use ranking algorithms to provide users with accurate and relevant results.

Applications of Ranking: - Information Retrieval - Collaborative Filtering - Automated Text Scoring (Essay Scoring) - Machine Translation - Sentence Parsing

Learning to Rank

Learning to rank or machine-learned ranking (MLR) is the application of machine learning, typically supervised, semi-supervised or reinforcement learning, in the construction of ranking models for information retrieval systems. Training data consists of lists of items with some partial order specified between items in each list. This order is typically induced by giving a numerical or ordinal score or a binary judgment (e.g. "relevant" or "not relevant") for each item. The ranking model purposes to rank, i.e. producing a permutation of items in new, unseen lists in a similar way to rankings in the training data.

Supervised Ranking

Pointwise model

Pointwise approaches look at a single document at a time in the loss function. They essentially take a single document and train a classifier / regressor on it to predict how relevant it is for the current query. The final ranking is achieved by simply sorting the result list by these document scores. For pointwise approaches, the score for each document is independent of the other documents that are in the result list for the query. All the standard regression and classification algorithms can be directly used for pointwise learning to rank.

Regression, Classification, Ordinal regression (items to be ranked are treated in isolation)

Scenario: We have customers searching for different movies and those have been ranked by a customer relevance score

► Code

	customer_id	movie	genre	price	rating	relevance
0	c4	movie_1	Animation	36.875232	0.109586	0.0
1	c2	movie_2	Horror	14.994330	29.543855	1.0
2	c2	movie_3	Crime	18.350202	24.308950	0.0
3	c5	movie_4	Children's	36.660083	15.435363	0.0
4	c1	movie_5	Comedy	14.498468	3.320721	0.0
...	...	...	...	...	...	...
995	c4	movie_996	Comedy	38.772438	59.713921	0.0
996	c3	movie_997	Thriller	36.178473	11.326334	0.0
997	c4	movie_998	Adventure	21.479971	43.585153	1.0
998	c1	movie_999	War	28.659169	0.708221	0.0
999	c5	movie_1000	Drama	15.476363	29.697476	1.0

1000 rows × 6 columns

Now that we have the data we can define the features we will use in our model and the target variable

► Code

Since we have a categorical variable we will dumify the data

► Code

then we can train/test split our dataset

► Code

so now we can train our classifier to predict how relevant is a movie for a customer

► Code

```
LogisticRegression()
```

let's take a copy of our test dataframe and apply the model

► Code

	price	rating	relevance	genre_Action	genre_Adventure	genre_Animation	genre_Children's	genre_Comec
0	44.963785	45.100109	0.0	0	0	0	0	0
1	21.442252	35.388441	1.0	0	0	0	0	0
2	30.280563	26.777554	0.0	0	0	0	0	0

3 rows × 23 columns

◀  ▶

We use the Compute Normalized Discounted Cumulative Gain(NDCG) to assess our model

Sum the true scores ranked in the order induced by the predicted scores, after applying a logarithmic discount.

Then divide by the best possible score (Ideal DCG, obtained for a perfect ranking) to obtain a score between 0 and 1.

► Code

```
0.9131585064991307
```

Pairwise model

Pairwise approaches look at a pair of documents at a time in the loss function. Given a pair of documents, they try and come up with the optimal ordering for that pair and compare it to the ground truth. The goal for the ranker is to minimize the number of inversions in ranking i.e. cases where the pair of results are in the wrong order relative to the ground truth. Pairwise approaches work better in practice than pointwise approaches because predicting relative order is closer to the nature of ranking than predicting class label or relevance score. Some of the most popular Learning to Rank algorithms like RankNet, LambdaRank and LambdaMART are pairwise approaches.

Rank-preference models (items to be ranked are treated in pairs)

Scenario: We have a lot of papers and we some very defined queries - we want to find the most relevant documents given the query

► Code

	qid	did	doc_size	doc_authors	doc_paragraphs	relevance	rank
499	query_1	doc_500	134.883309	5	127	0.094079	1
728	query_4	doc_729	43.758156	3	79	0.090593	1
727	query_2	doc_728	246.796883	3	101	0.147661	1
718	query_3	doc_719	348.961423	4	82	0.116877	1
715	query_3	doc_716	226.646396	4	146	0.157098	1
...	...	...	...	...	...	...	...
548	query_4	doc_549	15.807793	3	134	0.959892	5
835	query_4	doc_836	178.380919	5	112	0.829741	5
103	query_5	doc_104	215.429695	2	89	0.802579	5
113	query_5	doc_114	158.357579	5	136	0.891533	5
355	query_1	doc_356	276.640196	3	101	0.955163	5

1000 rows × 7 columns

again we define the features we are going to use in our models

► Code

We train test split the data, but also sort it by the rank we have assigned

► Code

in the next models we need to specify the number of samples related to group/query/domain

► Code

Actually, in Learning to Rank field, we are trying to predict the relative score for each document to a specific query. That is, this is not a regression problem or classification problem. Hence, if a document, attached to a query, gets a negative predict score, it means and only means that it's relatively less relative to the query, when comparing to other document(s), with positive scores.

Then we can train our XGBoost model using the [Pairwise](#) objective

► Code

```
XGBRanker(base_score=0.5, booster='gbtree', colsample_bylevel=1,
          colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
          importance_type='gain', interaction_constraints='',
          learning_rate=0.300000012, max_delta_step=0, max_depth=6,
          min_child_weight=1, missing=nan, monotone_constraints='()',
          n_estimators=100, n_jobs=32, num_parallel_tree=1, random_state=0,
          reg_alpha=0, reg_lambda=1, scale_pos_weight=None, subsample=1,
          tree_method='exact', validate_parameters=1, verbosity=None)
```

and predict the rank score using our model

► Code

Use subset (sliced data) of np.ndarray is not recommended because it will generate extra copies and increase memory consumption

	qid	did	doc_size	doc_authors	doc_paragraphs	relevance	rank	rank_score_pairwise
131	query_5	doc_132	438.245360	5	131	0.468934	3	5.788270
837	query_5	doc_838	491.327195	3	126	0.271177	2	5.774170
815	query_2	doc_816	38.272288	5	145	0.644796	4	5.751684
202	query_4	doc_203	120.329668	3	130	0.575423	3	5.630410
246	query_4	doc_247	421.376384	3	176	0.056486	1	5.495399
...	...	...	...	...	...	...	...	...
419	query_4	doc_420	358.623323	3	56	0.489202	3	-2.377359
917	query_5	doc_918	13.784377	3	75	0.943996	5	-2.487809
748	query_2	doc_749	497.286684	3	55	0.904371	5	-3.127292
282	query_4	doc_283	463.882003	3	57	0.680035	4	-3.233275
640	query_2	doc_641	114.068297	3	149	0.283694	2	-3.254235

330 rows × 8 columns

We are using again the NDCG score to see how our model performs

► Code

```
0.8948626084762222
```

► Code

	qid	did	doc_size	doc_authors	doc_paragraphs	relevance	rank	rank_score_pairwise
131	query_5	doc_132	438.245360	5	131	0.468934	3	5.788270
837	query_5	doc_838	491.327195	3	126	0.271177	2	5.774170
705	query_5	doc_706	463.690276	3	125	0.634287	4	5.332096
160	query_5	doc_161	48.579709	2	136	0.555408	3	4.670428
890	query_5	doc_891	202.299155	2	108	0.814885	5	4.618782
...	...	...	...	...	...	...	...	...
808	query_1	doc_809	424.648466	5	71	0.006189	1	-0.325586
850	query_1	doc_851	402.305516	4	68	0.554643	3	-0.591345
31	query_1	doc_32	470.012134	2	67	0.459052	3	-0.765779
857	query_1	doc_858	158.421303	2	177	0.703012	4	-1.140166
643	query_1	doc_644	160.910234	4	152	0.992542	5	-2.205011

330 rows × 8 columns

Listwise model

Listwise approaches directly look at the entire list of documents and try to come up with the optimal ordering for it.

There are 2 main sub-techniques for doing listwise Learning to Rank: - Direct optimization of IR measures such as NDCG. E.g. SoftRank, AdaRank - Minimize a loss function that is defined based on understanding the unique properties of the kind of ranking you are trying to achieve. E.g. ListNet, ListMLE

Treat each list as an instance. Usually tries to directly optimise the evaluation measure (e.g. mean average precision)

Using the same data as above we can train another model but this time using the [NDCG](#) objective

► Code

```
XGBRanker(base_score=0.5, booster='gbtree', colsample_bylevel=1,
           colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
           importance_type='gain', interaction_constraints='',
           learning_rate=0.300000012, max_delta_step=0, max_depth=6,
           min_child_weight=1, missing=nan, monotone_constraints='()'),
           n_estimators=100, n_jobs=32, num_parallel_tree=1,
           objective='rank:ndcg', random_state=0, reg_alpha=0, reg_lambda=1,
           scale_pos_weight=None, subsample=1, tree_method='exact',
           validate_parameters=1, verbosity=None)
```

and predict the ranking score using this model as well

► Code

Use subset (sliced data) of np.ndarray is not recommended because it will generate extra copies and increase memory consumption

	qid	did	doc_size	doc_authors	doc_paragraphs	relevance	rank	rank_score_pairwise	rank_score_ndcg
131	query_5	doc_132	438.245360	5	131	0.468934	3	5.788270	0.564546
100	query_4	doc_101	311.435717	4	115	0.777631	4	0.721314	0.564546
460	query_3	doc_461	339.116520	4	173	0.327702	2	3.660527	0.564546
705	query_5	doc_706	463.690276	3	125	0.634287	4	5.332096	0.564546
134	query_2	doc_135	422.214211	4	170	0.902875	5	4.722393	0.564546
...	...	...	...	...	...	...	...	...	...
421	query_5	doc_422	131.493036	2	69	0.524366	3	3.481098	0.432135
194	query_4	doc_195	86.266093	2	102	0.499842	3	4.688314	0.432135
5	query_1	doc_6	154.925980	4	94	0.500868	3	1.975777	0.432135

	qid	did	doc_size	doc_authors	doc_paragraphs	relevance	rank	rank_score_pairwise	rank_score_ndcg
663	query_3	doc_664	139.543270	5	61	0.438917	3	3.006506	0.432135
528	query_5	doc_529	130.788445	2	92	0.521378	3	4.387608	0.432135

330 rows × 9 columns

We are using again the NDCG score to see how our model performs

► Code

0.8841611992803472

► Code

	qid	did	doc_size	doc_authors	doc_paragraphs	relevance	rank	rank_score_pairwise	rank_score_ndcg	
	131	query_5	doc_132	438.245360	5	131	0.468934	3	5.788270	0.564546
	705	query_5	doc_706	463.690276	3	125	0.634287	4	5.332096	0.564546
	647	query_5	doc_648	499.702704	5	112	0.640742	4	3.949763	0.564546
	37	query_5	doc_38	446.435847	2	139	0.078024	1	1.788967	0.564546
	372	query_5	doc_373	376.365562	4	118	0.256528	2	1.769992	0.564546

...	...	...	...	...	...	...	...	...	...
412	query_1	doc_413	146.237719	3	101	0.505013	3	3.511979	0.432135
593	query_1	doc_594	51.742807	5	50	0.408177	3	2.817086	0.432135
174	query_1	doc_175	152.116111	4	103	0.077273	1	2.433534	0.432135
518	query_1	doc_519	116.540539	5	103	0.418459	3	4.447410	0.432135
5	query_1	doc_6	154.925980	4	94	0.500868	3	1.975777	0.432135

330 rows × 9 columns

4									
---	--	--	--	--	--	--	--	--	--

Expedia Hotel Recommendation



Expedia has provided you [logs of customer behavior](#). These include what customers searched for, how they interacted with search results (click/book), whether or not the search result was a travel package. The data in this competition is a random selection from Expedia and is not representative of the overall statistics.

Expedia is interested in predicting which hotel group a user is going to book. Expedia has in-house algorithms to form hotel clusters, where similar hotels for a search (based on historical price, customer star ratings, geographical locations relative to city center, etc) are grouped together. These hotel clusters serve as good identifiers to which types of hotels people are going to book, while avoiding outliers such as new hotels that don't have historical data.

Column name	Description	Data type
date_time	Timestamp	string
site_name	ID of the Expedia point of sale (i.e. Expedia.com, Expedia.co.uk, Expedia.co.jp, ...)	int
posa_continent	ID of continent associated with site_name	int
user_location_country	The ID of the country the customer is located	int
user_location_region	The ID of the region the customer is located	int
user_location_city	The ID of the city the customer is located	int
orig_destination_distance	Physical distance between a hotel and a customer at the time of search. A null means the distance could not be calculated	double
user_id	ID of user	int
is_mobile	1 when a user connected from a mobile device, 0 otherwise	tinyint
is_package	1 if the click/booking was generated as a part of a package (i.e. combined with a flight), 0 otherwise	int
channel	ID of a marketing channel	int
srch_ci	Checkin date	string
srch_co	Checkout date	string

Column name	Description	Data type
srch_adults_cnt	The number of adults specified in the hotel room	int
srch_children_cnt	The number of (extra occupancy) children specified in the hotel room	int
srch_rm_cnt	The number of hotel rooms specified in the search	int
srch_destination_id	ID of the destination where the hotel search was performed	int
srch_destination_type_id	Type of destination	int
hotel_continent	Hotel continent	int
hotel_country	Hotel country	int
hotel_market	Hotel market	int
is_booking	1 if a booking, 0 if a click	tinyint
cnt	Numer of similar events in the context of the same user session	bigint
hotel_cluster	ID of a hotel cluster	int

► Code

	date_time	site_name	posa_continent	user_location_country	user_location_region	user_location_city	orig_destination_dis
0	2014-08-11 07:46:59	2	3	66	348	48862	2234.2641
1	2014-08-11 08:22:12	2	3	66	348	48862	2234.2641
2	2014-08-11 08:24:33	2	3	66	348	48862	2234.2641
3	2014-08-09 18:05:16	2	3	66	442	35390	913.1932
4	2014-08-09 18:08:18	2	3	66	442	35390	913.6259

5 rows × 24 columns

◀		▶
---	--	---

Modeling

Our objective here is to predict a list of hotel clusters for each user.

But first let's make a list of features and a variable for target and group_id

we use group_id to define the column that will count the number of samples related to a group/query/domain

► Code

we use this fuction for feature engineering - we want to take the number of the mount from the check in search date - we want to extract the number of days the user is looking to book

► Code

we apply the feature engineering function

► Code

after that we can train/test split our data

► Code

then we want to get the number of samples by our group_id column in this case is the `used_id` column

Note: the sum of the number of groups must match the size of the training set

► Code

► Code

6773

finally we can train our ranking model

► Code

```
XGBRanker(base_score=0.5, booster='gbtree', colsample_bylevel=1,
           colsample_bynode=1, colsample_bytree=1, gamma=0, gpu_id=-1,
           importance_type='gain', interaction_constraints='',
           learning_rate=0.300000012, max_delta_step=0, max_depth=6,
           min_child_weight=1, missing=nan, monotone_constraints='()'),
           n_estimators=100, n_jobs=32, num_parallel_tree=1,
           objective='rank:ndcg', random_state=0, reg_alpha=0, reg_lambda=1,
           scale_pos_weight=None, subsample=1, tree_method='exact',
           validate_parameters=1, verbosity=None)
```

Now that we've trained our model we can take a copy of our test data and apply the model.

Then we predict the relevance score for each group in our dataframe.

► Code

► Code

	date_time	site_name	posa_continent	user_location_country	user_location_region	user_location_city	orig_destinatio
11832	2014-12-03 23:07:59	2	3	66	174	5013	4134.9107
57880	2013-03-02 17:59:35	2	3	66	318	7412	6007.6063

	date_time	site_name	posa_continent	user_location_country	user_location_region	user_location_city	orig_destinatio
165013	2013-03-18 23:49:34	30	4	195	995	55763	NaN
165016	2013-03-18 23:55:09	30	4	195	995	55763	NaN
14238	2014-01-08 14:19:07	13	1	46	171	56407	8597.2618

5 rows × 28 columns

Again we use the NDCG score to see how our model performs

► Code

0.9546739271914936

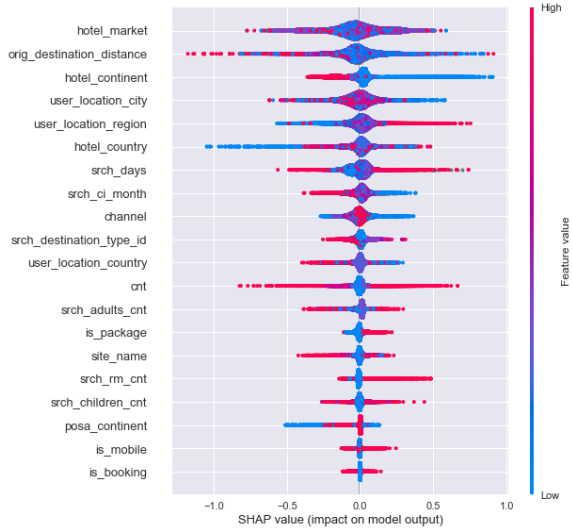
Now let's look at the feature importance and impact

► Code

```
ntree_limit is deprecated, use `iteration_range` or model slicing instead.
```

At first glance we can see: - a low value/category for hotel_continent is increasing the relevance score where a high value is decreasing it - on the other hand a high value/category for user location region is increasing the score and the reverse of it when the feature value is low - many features have a contradictoy impact whre a high value is having a both negative and positive impact toward the model output

► Code



Conclusions

- Learning to rank is another subbranch of machine learning focused on ranking and usually used in Information Retrieval (IR)
- Very large datasets are used in training the models in this area, but in the same time filtering capabilities are serving an important role
- Framing the problem right give **data** and **goal** is an essential part for solving ranking problem

References

- [Tiered machine learned ranking improves relevance for the retail search](#)
- [Learning to Rank](#)
- [learning-to-rank-1](#)
- [Learning to Rank with Partially-Labeled Data](#)
- [Learning to Rank with XGBoost and GPU](#)
- [Ranking Ads with Machine Learning](#)
- [How we've reworked our listings at OLX](#)
- [Notes on the NDCG metric used in the Visual Dialog Challenge 2019](#)
- [Learning to Rank with Partially-Labeled Data](#)